Presentation Narration Script

Greedy / Nearest Neighbor TSP — 14-slide presentation. Suggested duration: approximately 13–14 minutes including the live demonstration.

01 • Title (0:40)

On-screen action: Open with the title slide; identify the assigned method.

Good day. Our presentation is about the Traveling Salesperson Problem, or TSP, and its application to a simple landmark tour in Cagayan de Oro City. We were assigned the Greedy Algorithm approach, specifically the nearest-neighbor procedure. We will explain the method, construct a weighted graph from real landmark coordinates, execute our Python program using the same data, and interpret the resulting route.

02 • Real-world problem (0:55)

On-screen action: Point to the tour scenario and the requirement to return home.

Imagine a visitor who wants to photograph and document several landmarks around downtown Cagayan de Oro. The visitor starts at Gaston Park, visits every required destination once, and returns to Gaston Park. The question is: in what order should the visitor make these stops to reduce the total distance? This is a form of the Traveling Salesperson Problem. In our implementation, the objective is to minimize the total modeled distance of a closed tour.

03 • Five landmarks (1:00)

On-screen action: Read the labels A to E. Emphasize that Gaston Park is the starting point.

We chose five real locations. Vertex A is Gaston Park, our starting and ending point. Vertex B is Plaza Divisoria. Vertex C is Ayala Malls Centrio. Vertex D is SM CDO Downtown. Vertex E is Limketkai Mall. We use approximate published latitude and longitude coordinates for these landmarks. These coordinates and their references are included in the dataset, so someone reviewing the work can check where each location comes from.

04 • Graph and assumptions (1:10)

On-screen action: Explain complete, undirected and why these are proxies.

A weighted graph consists of vertices, edges and weights. In our graph, vertices represent landmarks. Edges represent a possible direct transition from one landmark to another, and weights are distances in kilometers. We chose a complete, undirected graph, meaning any pair is connected and the distance from A to B equals the distance from B to A. We calculate straight-line distances from the published coordinates using the Haversine formula and round each edge weight to three decimal places. Importantly, these values are not actual Google Maps road or walking distances. The simplified model ignores road layout, barriers, traffic, and building entrances.

05 • Input matrix (0:55)

On-screen action: Point to B→C and to the diagonal zeros.

The distance matrix gives all weights used by the algorithm. Every row and column has a location label, from A to E. As one example, B to C is one point one five seven kilometers. The diagonal is zero because traveling from a location to itself has zero distance. The matrix is symmetric because we use straight-line geometry, not directed street travel. This exact matrix appears in our CSV input and is read by the Python implementation.

06 • Graph illustration (0:55)

On-screen action: Explain thin candidate connections and gold chosen edges.

Here is the labeled graph based on the approximate coordinates. The thin connections show the complete graph: every location could theoretically connect to every other location. The highlighted arrows show the tour produced by Greedy. The image is a geographic schematic rather than a street map. The values in the graph correspond to the same distance matrix shown previously, so the visual model and computational model stay consistent.

07 • Greedy procedure (1:20)

On-screen action: Read the pseudocode in logical groups, not line by line.

The greedy approach chooses the best immediate move available. For our assigned nearest-neighbor algorithm, we begin at a chosen location and mark it as visited. While unvisited locations remain, we compare the distances from our current location to those unvisited vertices and select the nearest one. We then mark that new location as visited and repeat. After all locations are visited, we return to the starting vertex. The algorithm returns the complete closed tour and its total distance. It is simple, but a locally shortest choice does not always produce a globally shortest tour.

08 • Manual decisions (1:10)

On-screen action: Trace each move on the table. State the five leg distances.

Let us manually follow the algorithm. Starting at A, the nearest unvisited location is B at zero point three one six kilometers. From B, the nearest unvisited destination is C at one point one five seven kilometers. From C, we choose D at zero point three five three kilometers. D next chooses E at zero point three six five kilometers. Once every location has been visited, we return from E to A, which is one point seven four six kilometers. Adding these five matrix values gives a total of three point nine three seven kilometers.

09 • Code structure (0:50)

On-screen action: Explain the three stages of the code at a high level.

Our Python program has three main parts. First, it reads the locations and distance matrix from CSV files and checks that the input labels and weights are consistent. Second, the nearest-neighbor function keeps track of visited and unvisited vertices and repeatedly selects the smallest eligible edge. Third, a validation function checks that the tour starts and ends at A, that all five locations occur exactly once before returning, and that the output total matches the sum of the selected edge weights. The program does not require extra libraries.

10 • Live Python run (2:05)

On-screen action: IMPORTANT: switch to terminal. Run Python live, scroll through sections, return to slides.

We will now run the program using our application data. We open the project folder, where the Python file and the two CSV input files are stored. Then we type python TSP_Greedy_Implementation.py and press Enter. At the top of the output, you can see the five landmark labels and the input distance matrix. Next, the program displays the successive greedy decisions: A to B, B to C, C to D, and D to E, followed by the return to A. The result is displayed as both the letter sequence and the full landmark names. Below that, the program recalculates each leg of the route. The output confirms that each required location was visited exactly once and that the route returned to Gaston Park. The total is three point nine three seven kilometers. These are straight-line model distances, not the lengths of pedestrian routes on Google Maps.

11 • Results and validation (0:55)

On-screen action: Show the route and 3.937 km again; identify completed TSP requirements.

Our result is the closed route A to B to C to D to E and back to A. It starts and ends at Gaston Park and visits each required location once. The independent leg-by-leg sum is three point nine three seven kilometers, exactly the amount reported by the Python program. This demonstrates that the Greedy implementation correctly applies the input graph and produces a valid solution to the problem as we modeled it.

12 • Optimality and complexity (1:00)

On-screen action: Be precise: Greedy is not generally optimal; optional exhaustive check only for this small dataset.

Nearest Neighbor is a heuristic and does not guarantee the globally shortest tour in every Traveling Salesperson Problem. With n vertices, the algorithm may inspect up to n minus one unvisited vertices in a step. Repeating those scans gives a time complexity of O of n squared, as stated in our assigned reference. Because our dataset has only five vertices, we also ran a separate exhaustive audit of the four factorial, or twenty-four, possible fixed-start tours. On this particular matrix, the minimum was also three point nine three seven kilometers. This supports optimality only for the given dataset, not as a universal property of Greedy.

13 • Conclusion (0:55)

On-screen action: Summarize the solution and one realistic improvement.

In conclusion, we represented a five-location city tour as a weighted Traveling Salesperson Problem, followed the assigned greedy nearest-neighbor algorithm, and implemented it in Python. The program found a valid closed route with a modeled distance of three point nine three seven kilometers. The main limitation is that our Haversine distances are straight-line estimates. A practical improvement would be to replace them with verified walking or driving distances from Google Maps, accounting for real paths and accessibility. Thank you.

14 • References (0:20)

On-screen action: End on the bibliography. Click Google Maps route link if useful; do not substitute its travel distance.

These are our sources for the greedy procedure and for the published location coordinates. We also include a Google Maps route link for geographic context and verification. The numeric weights in our graph come from the coordinate calculations, not Google Maps directions.